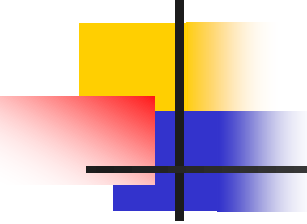


搜索

张长水

清华大学 自动化系

zcs@mail.tsinghua.edu.cn

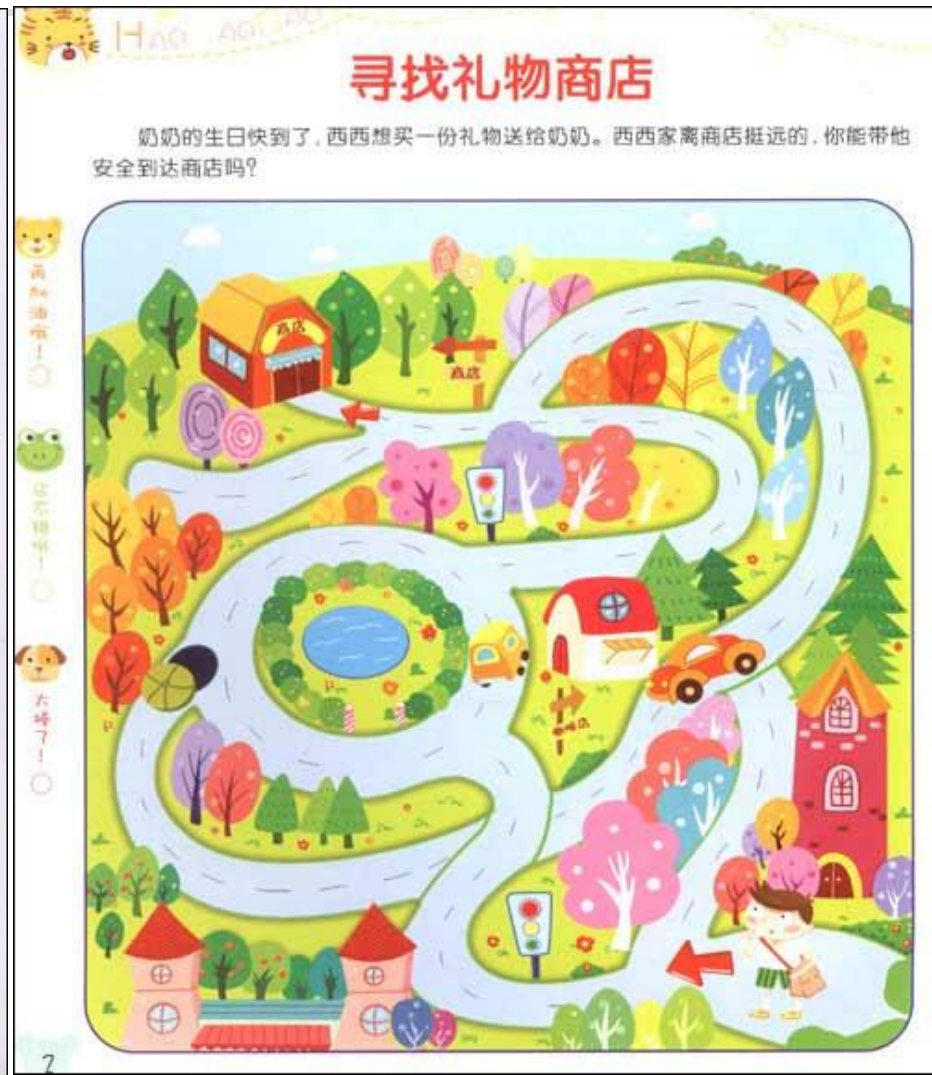


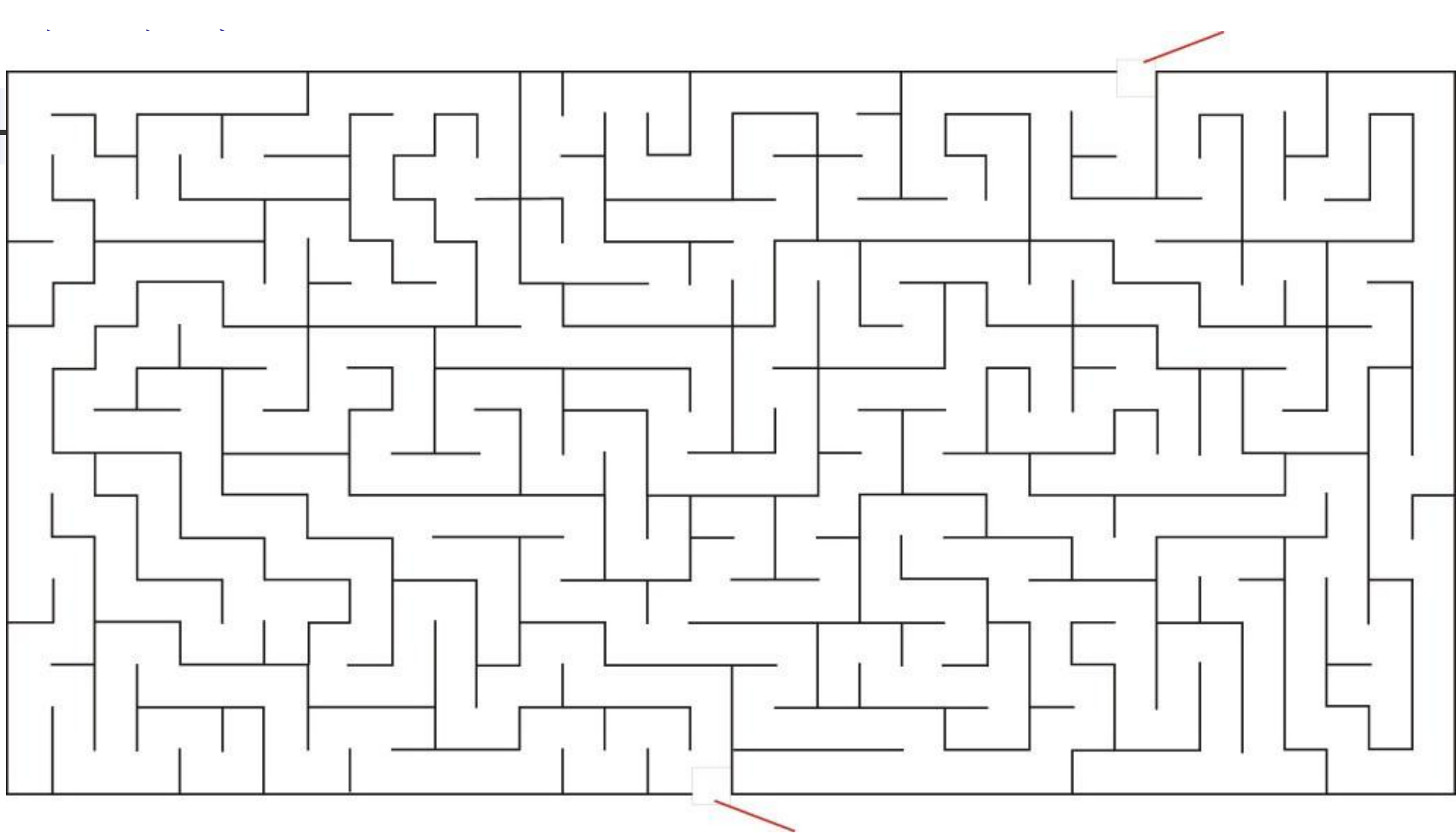
问题求解

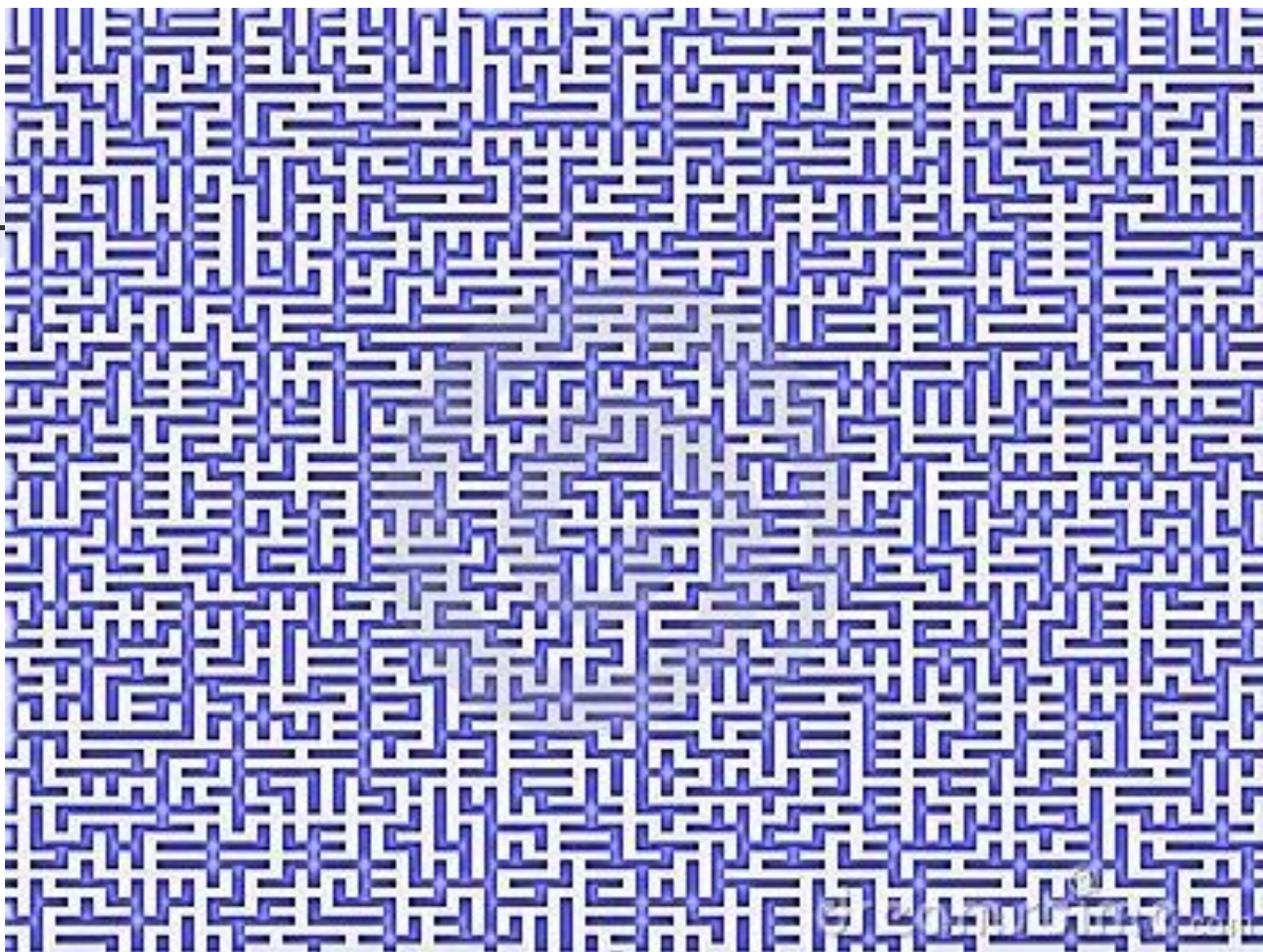
- 现实中很多智能问题有不同表现，其背后共同的问题是什么？
- 问题求解(**Problem Solving**)是人工智能第一个大的成就。许多的问题求解工作可以通过**搜索**得到解决

走迷宫

- 为什么研究走迷宫？
 - 简单、直观
 - 具有代表性



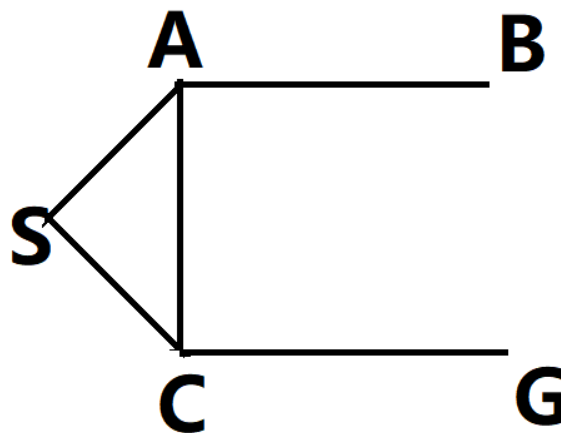
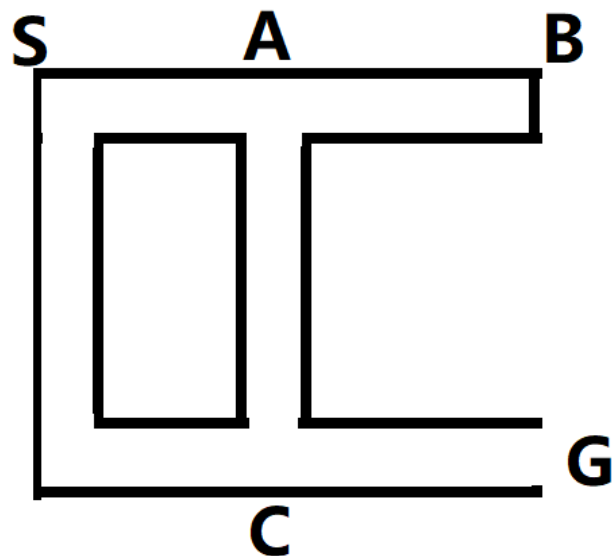




表示一个求解问题

搜索技术的思路

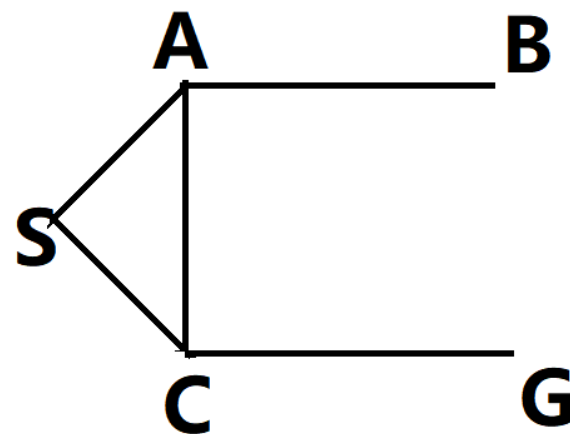
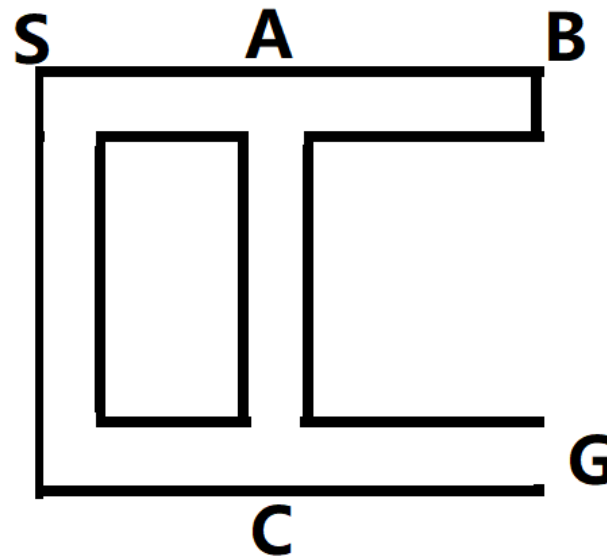
- 表示一个求解问题
- 给出一个搜索算法



表示一个求解问题

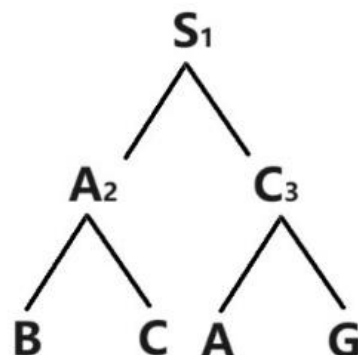
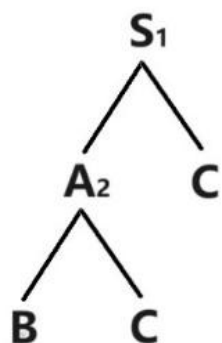
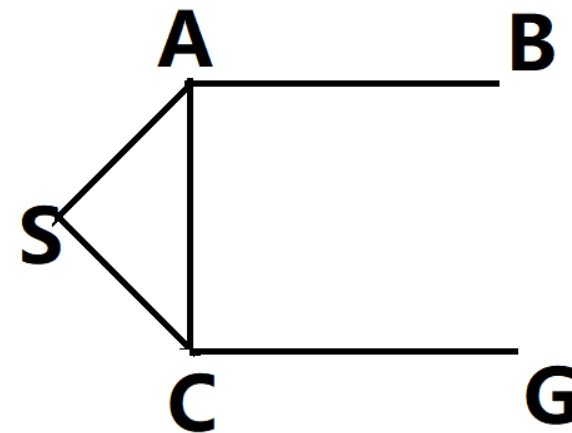
搜索技术的思路

- 每个节点
- 节点之间的连线
- 用图 (graph) (V, E) 表示
- 节点集合 $V = \{S, A, B, C, G\}$
- 边的集合是 $E = \{(S, A), (S, C), (A, B), (A, C), (C, G)\}$



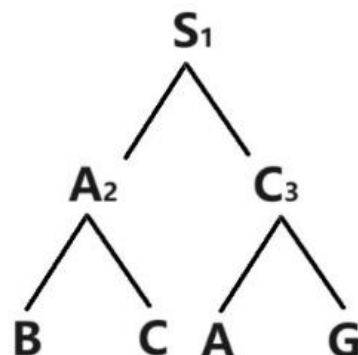
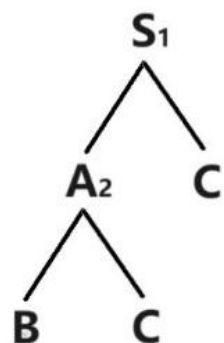
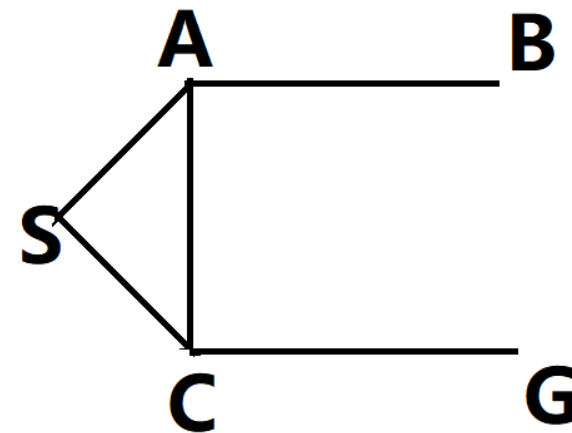
给出一个搜索算法

- 算法：在什么情况下该做什么
- 一个具体的算法
- 树结构
- 宽度优先搜索算法



给出一个搜索算法

- 算法：在什么情况下该做什么
- 一个算法：搜索过程，节点顺序号
- 树结构
- 宽度优先搜索算法



宽度优先搜索算法:

(1) 把起始节点 S 放到 Open 表中 (Open 表是一个先进先出的堆栈), 如果该起始节点为一目标节点, 则得到一个解, 算法结束。

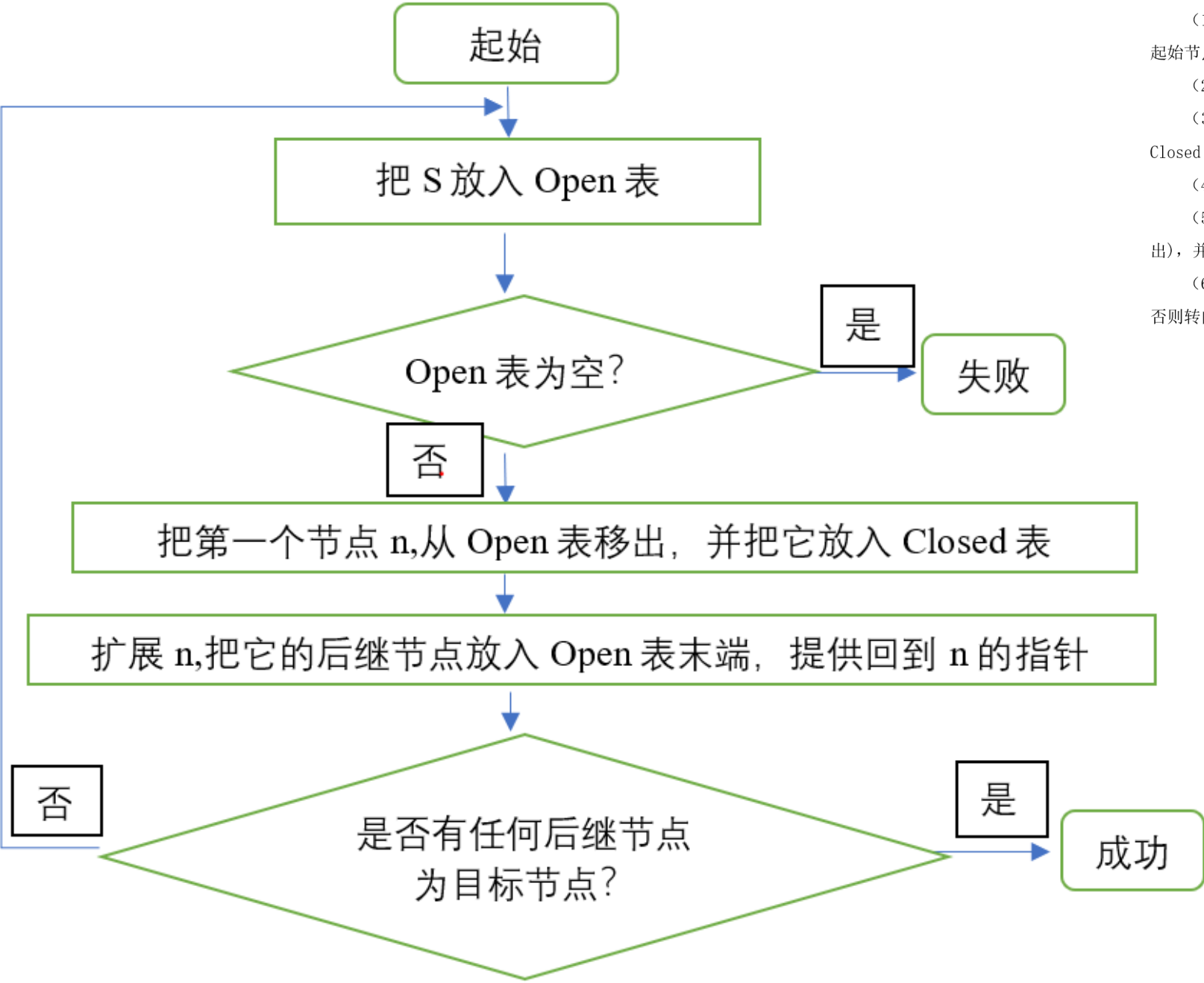
(2) 如果 Open 是个空表, 则没有解, 失败退出; 否则继续。

(3) 把 Open 表的第一个节点 (节点 n) 移出, 并把它放入 Closed 表中。
Closed 表是一个的扩展过的节点表。

(4) 扩展节点 n 。如果 n 没有后继节点, 则转向步骤 (2)。

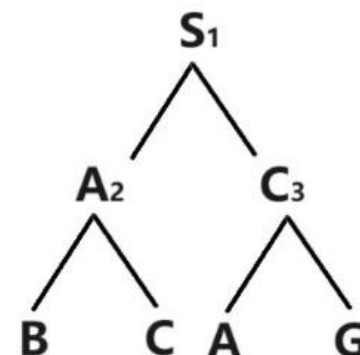
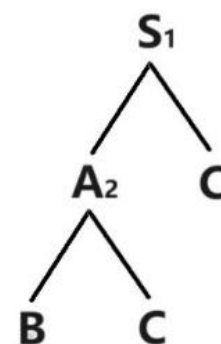
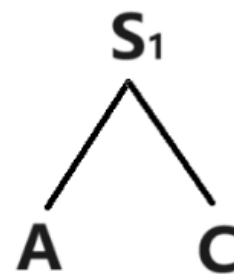
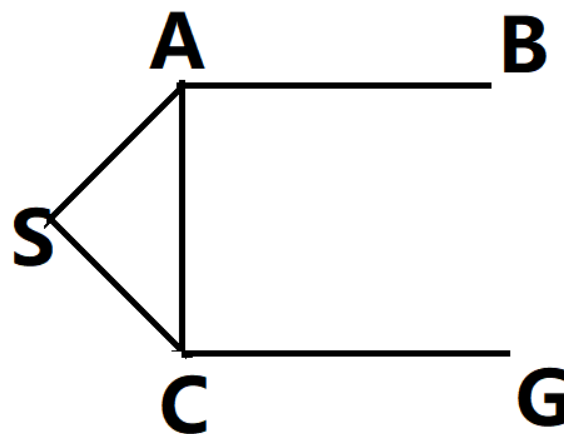
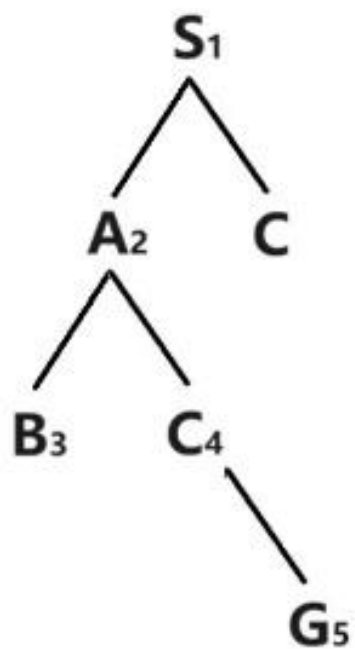
(5) 把 n 的所有后继节点放到 Open 表的末端 (末端的节点比前端的节点后移出), 并提供从这些后继节点指向 n 的指针。

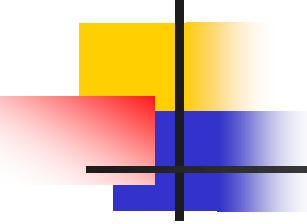
(6) 如果 n 的任一后继节点是一个目标节点, 则找到一个解, 成功退出; 否则转向步骤 (2)。



另一个搜索算法

- 深度优先搜索算法





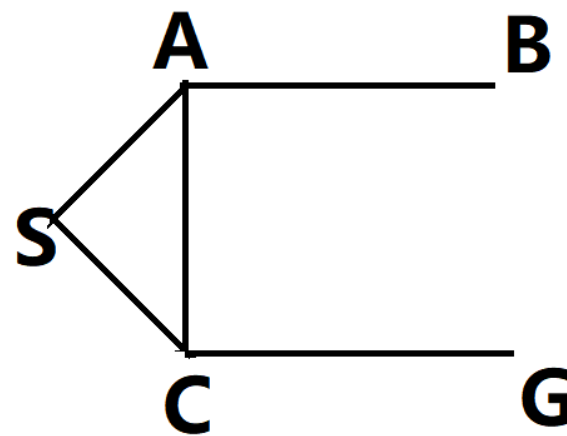
都有哪些搜索算法？

- 宽度优先搜索
- 深度优先搜索
- 等费用方法
- 爬山法
- A算法
- **A*算法**：搜索算法的一个高峰

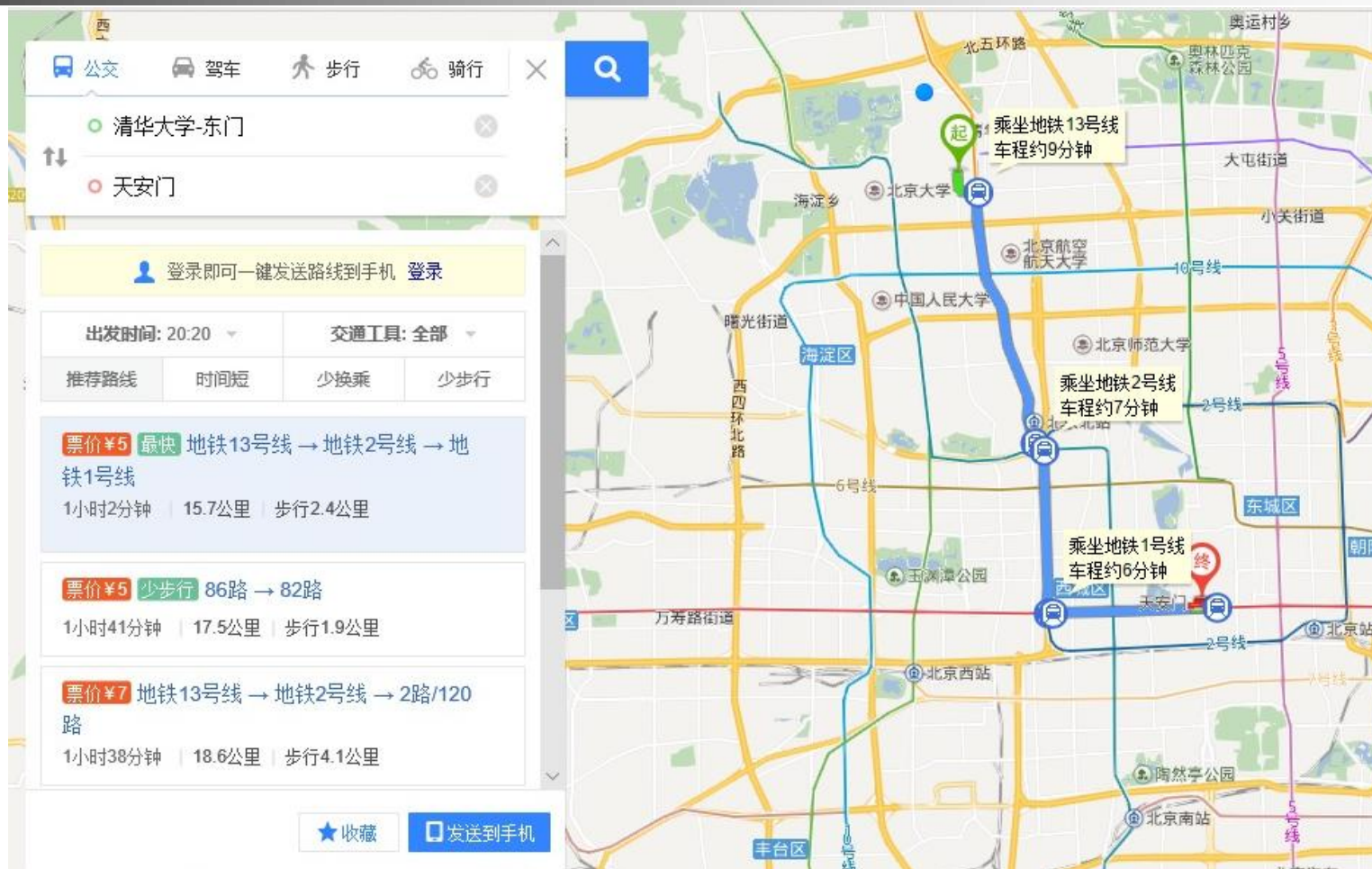
理论分析

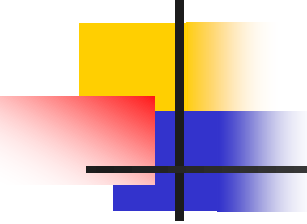
搜索算法的性质

- 算法能找到问题的解吗？（连通图可以）
- 算法能找到问题的最优解吗？（有的可以）
- 算法快吗？**计算复杂性**如何？



搜索算法应用举例



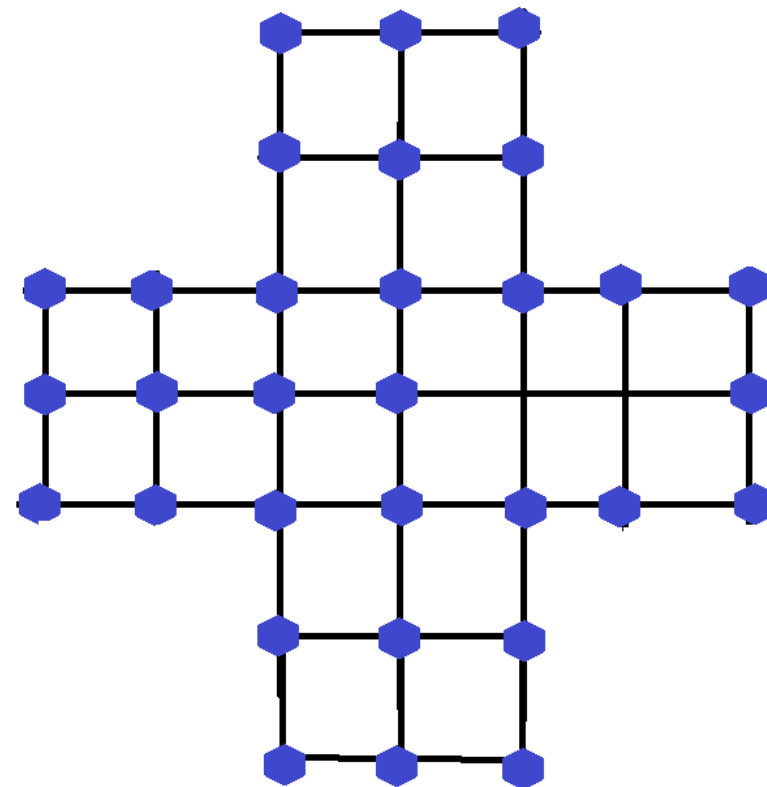
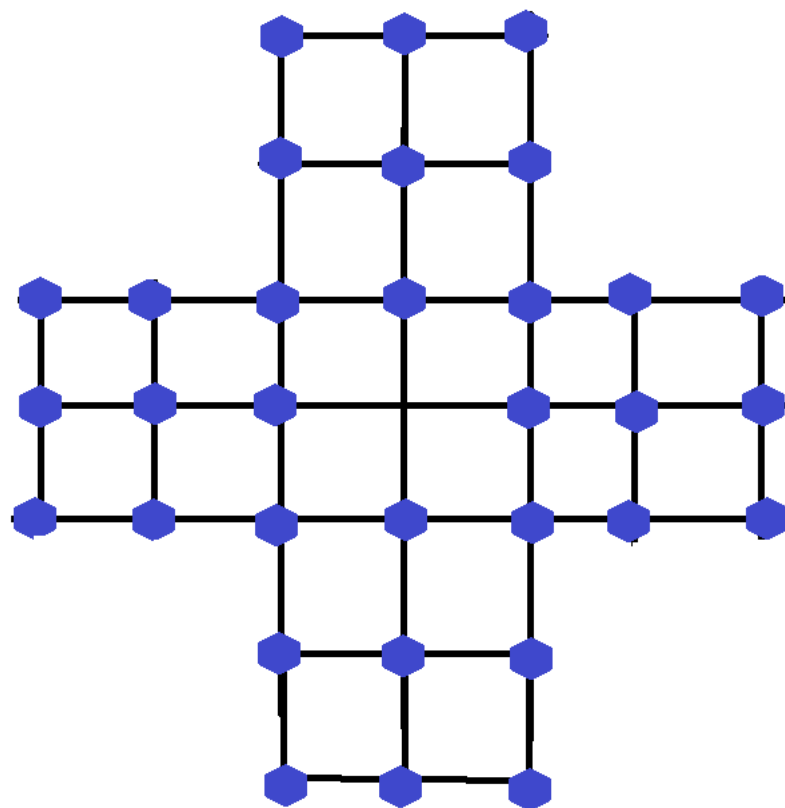


搜索算法应用举例

- 独立钻石游戏规则

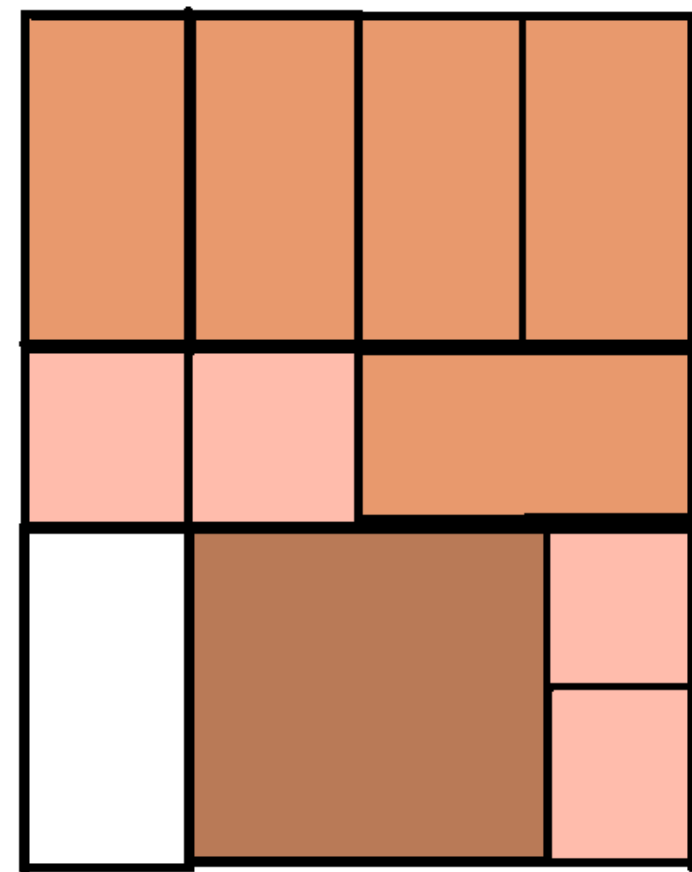
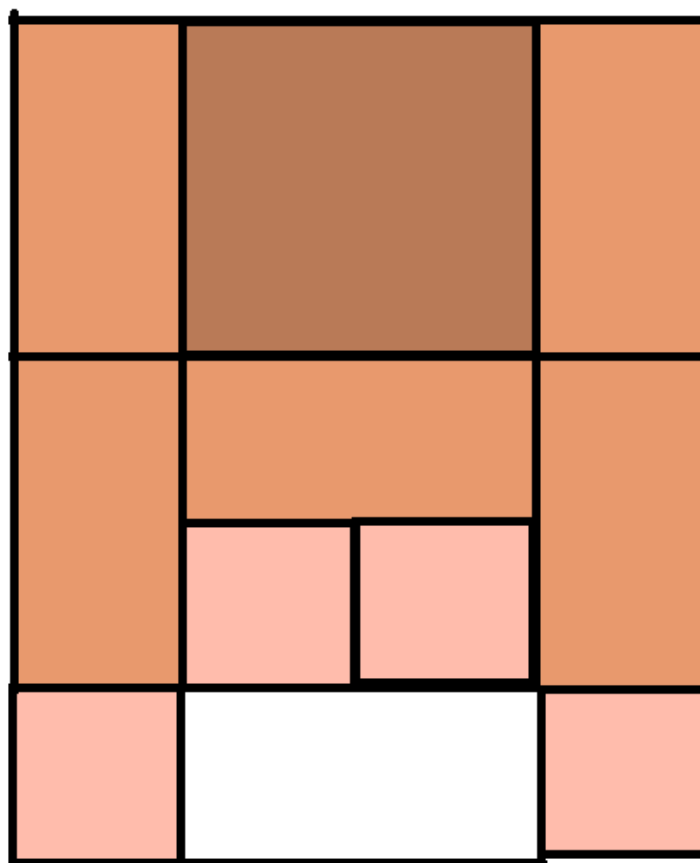
- 什么是其状态？

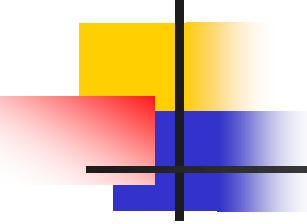
- 什么是分支？



搜索算法应用举例

- 捉放曹游戏规则
- 什么是其状态?
- 什么是分支?





搜索算法应用举例

- 重排九宫游戏规则
- 什么是其状态?
- 什么是分叉?

4	6	1
	8	2
3	7	5

1	2	3
8		4
7	6	5

2	1	3
8		4
7	6	5

搜索算法应用举例

- 启发函数与启发式搜索

- $f(n)=g(n)+h(n)$

- $h(n)=P(n)+3S(n)$

- $g(n)$: 从初始状态出发到当前状态所移动过的牌的次数

- 每一张牌*i*与其目标位置的距离为*d(i)*，8张牌所有*d(i)*求和： $P(n)$

- 计算逆序数，每个逆序算2分。中心有牌记分1，没有牌记分0；所有牌的分数求和： $S(n)$

4	6	1
	8	2
3	7	5

1	2	3
8		4
7	6	5

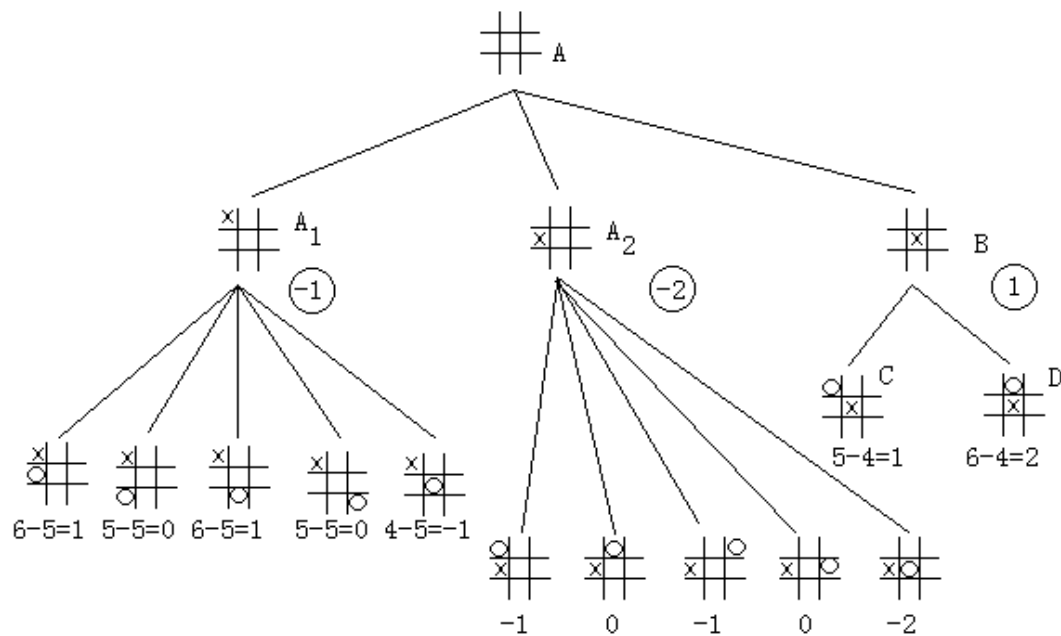
2	1	3
8		4
7	6	5

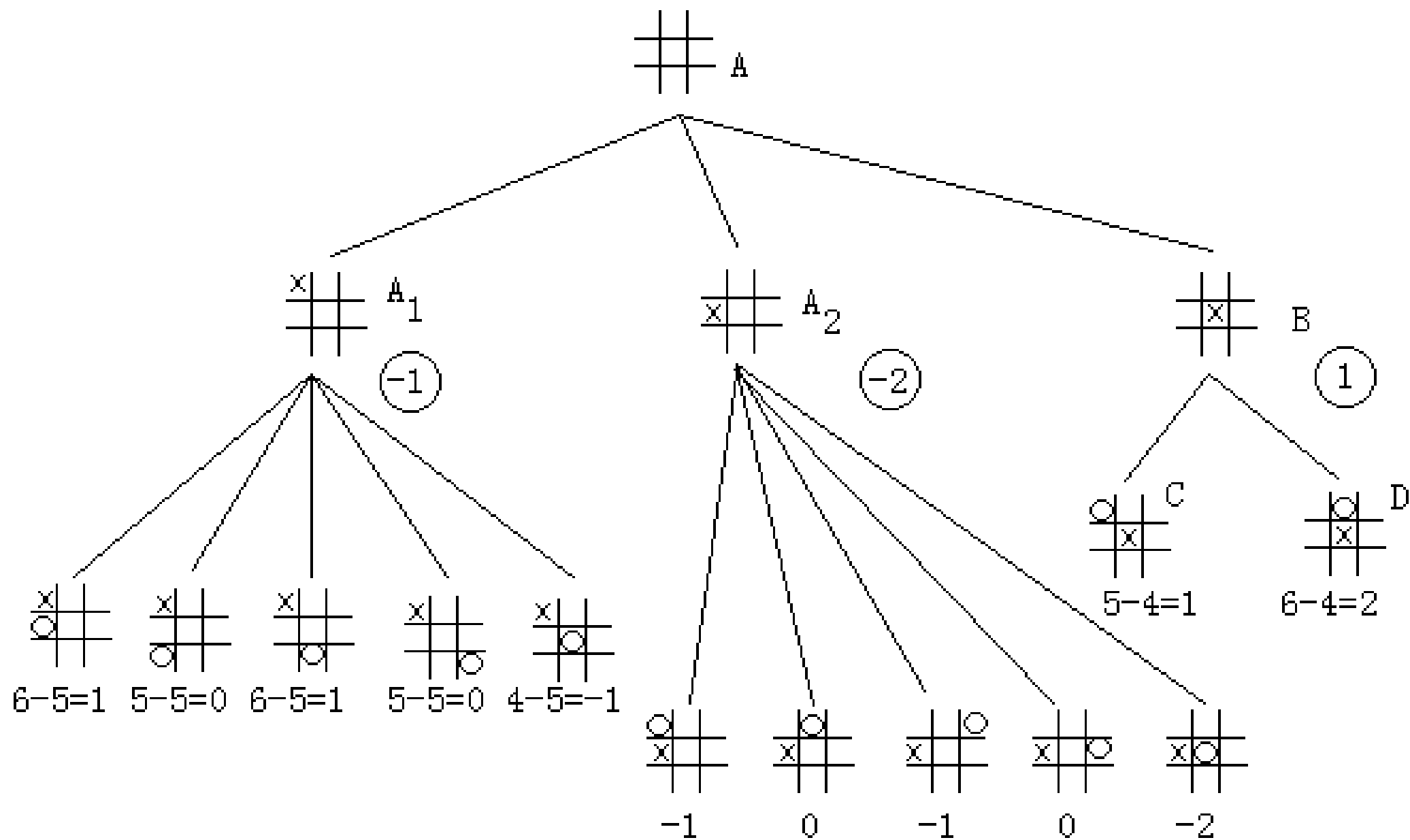
搜索算法可以用于下棋

下棋-博弈 (Game)

- 两人对垒，轮流走步。
- 双方走棋历史彼此都知道，下一步所有可能的走棋彼此也知道

- 每一方都选择对自己最有利的方案
- 最大最小搜索 (min-max search)





计算机下棋

- 跳棋
- 亚瑟.塞缪尔 (Arthur Samuel)
- 1952年在IBM研发了跳棋程序
- 启发式搜索
- 1962年胜了美国康涅狄格州的跳棋冠军



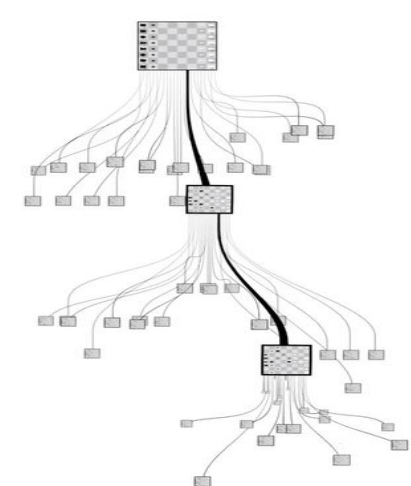
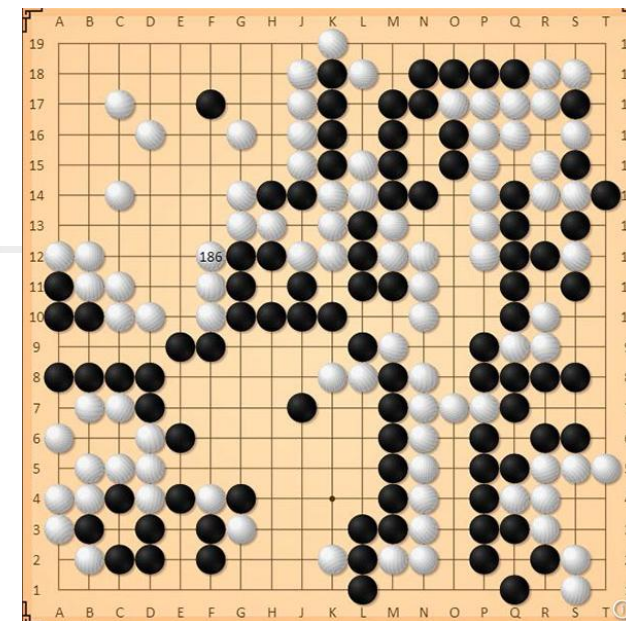
计算机下棋

- 国际象棋
- Deep Thought 2: 1991, 悉尼, 与澳大利亚冠军比赛, 1:1。
- Deep Blue: 1996. 2:4 输卡斯帕罗夫。
- Deeper Blue: 1997. 3.5:2.5 胜卡斯帕罗夫



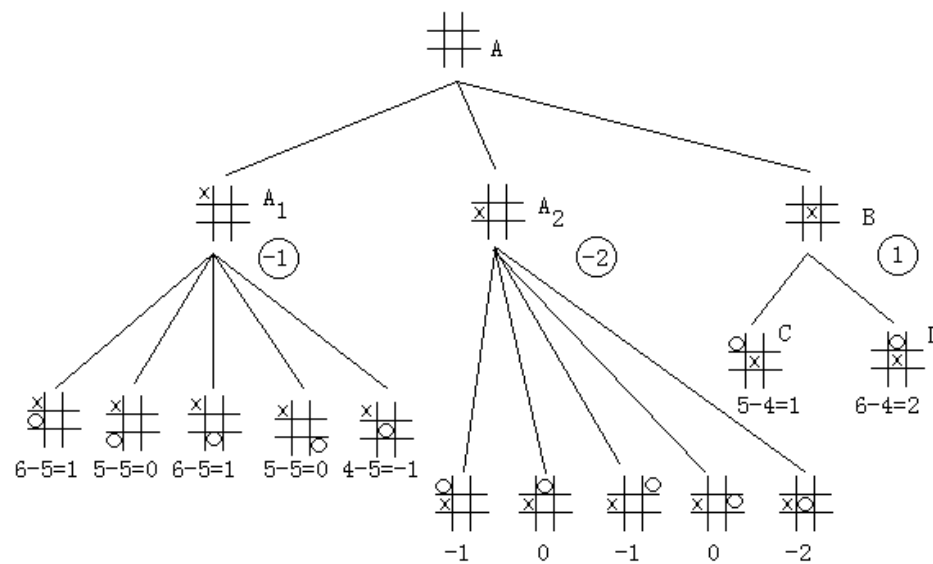
计算机下棋

- 围棋
- AlphaGo 战胜了围棋大师（2016年）
- 深度学习
- 强化学习



搜索算法的困难

- 计算量太大（太大的搜索空间，计算复杂性）
- 每个分叉点有太多的选择
- 围棋、象棋



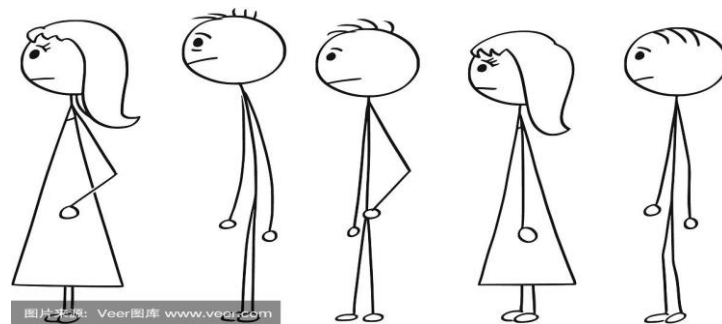
一个问题

例：一群人比年龄

```
for (i=0; i<n; i++)
```

...;

计算规模为n （n：问题规模）



图片来自百度

第二个问题

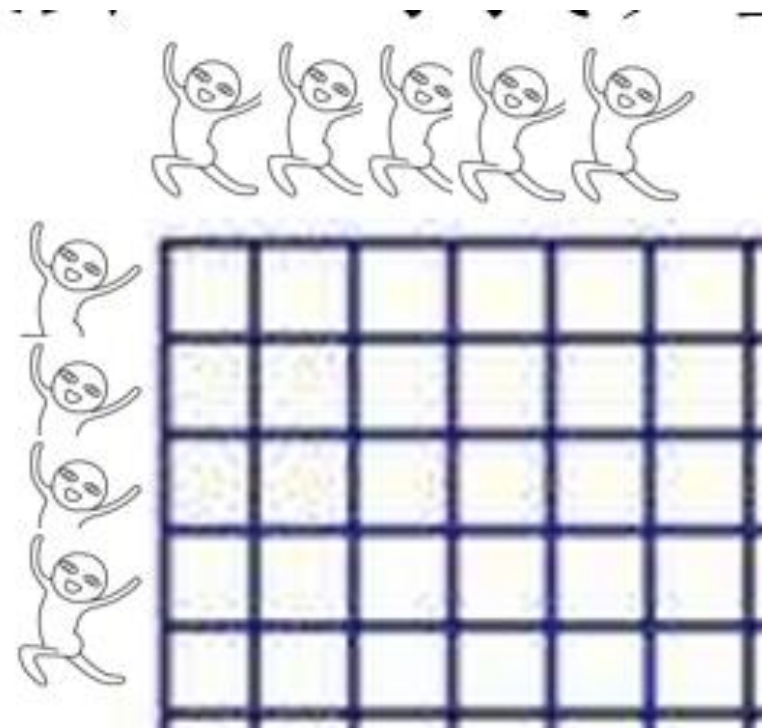
例：一群人找相同生日

```
for (i=0; i<n; i++)
```

```
for (j=0; j<n; j++)
```

```
k=i+j;
```

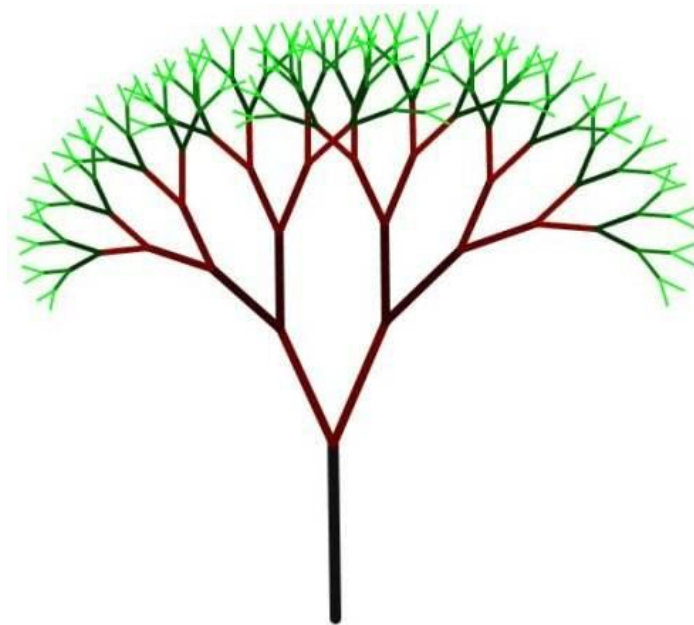
计算规模为 n^2 (n: 问题规模)

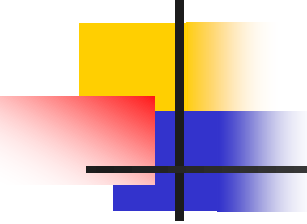


第三个问题

例. 一群N个人中的一组人工作

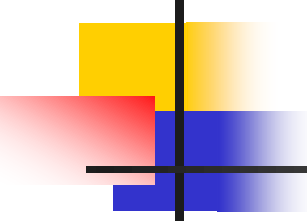
深度为n的二叉树，要访问所有结点。计算规模为： 2^n
(n: 问题规模)





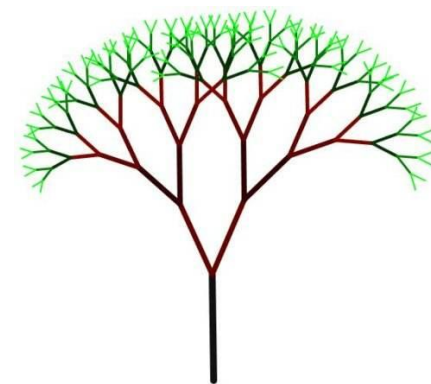
指数爆炸

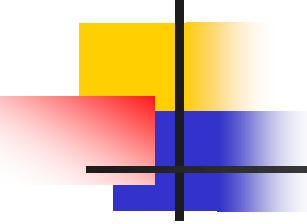
	10	20	30	40
n	0.00001秒	0.00002秒	0.00003秒	0.00004秒
n^2	0.0001秒	0.0004秒	0.0009秒	0.0016秒
2^n	0.001秒	1.0秒	17.9分	12.7天
3^n	0.059秒	58分	6.5年	3855世纪



提高计算机速度与可解决问题的关系

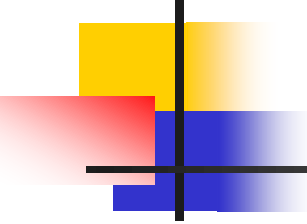
	用现在的计算机	快100倍	快1000倍
n	$N1$	$100N1$	$1000N1$
n^2	$N2$	$10N2$	$31.6N2$
2^n	$N3$	$N3+6.64$	$N3+9.97$
3^n	$N4$	$N4+4.19$	$N4+6.19$





使用知识

- 指数爆炸是一个根本性的难题
- 人工智能很多研究都卡在这个地方
- 使用知识：人工智能的一个原则
- 每一个问题具有其特殊性
- 启发式信息的使用：如重排九宫



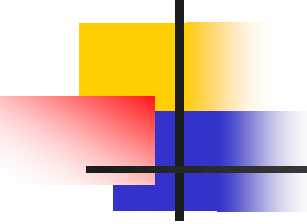
如何得到一个好的启发式函数？

设计一个启发式函数

- 需要对要解决的问题非常熟悉

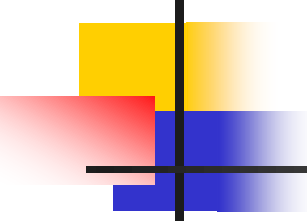
学习一个启发式函数

- 从大量的数据中学习（正在研究中）



知识点

- 如何表示一个问题
- 如何通过搜索求解一个问题
- 算法可能具有不同的性质
- 计算量的指数爆炸
- 人工智能一个原则：利用知识



问题？
